MissGNN: Missingness-Guided Graph Neural Network for Feature- and Sample-Aware Imputation

Zhongyi Yu[1], Zhenghao Wu[2], Shuhan Zhong[3], Weifeng Su[3,*], Amelie Chi Zhou[4], S.-H. Gary Chan[5], Chul-Ho Lee[6], and Weipeng Zhuo[3,*]

[1]School of Electrical Engineering and Computer Science, The University of Queensland
[2]University College Dublin [3]Beijing Normal-Hong Kong Baptist University
[4]Hong Kong Baptist University [5]The Hong Kong University of Science and Technology [6]Texas State University
zhongyi.yu@uq.edu.au, zhenghao.wu@ucdconnect.ie, {shuhanzhong, wfsu, weipengzhuo}@bnbu.edu.cn,
amelieczhou@comp.hkbu.edu.hk, gchan@cse.ust.hk, chulho.lee@txstate.edu

***Abstract*—Missing values are a common problem that poses significant challenges to data analysis and machine learning. This problem necessitates the development of an effective imputation method to fill in the missing values accurately, thereby enhancing the overall quality and utility of the datasets. Existing imputation methods, however, fall short of explicitly considering the 'missingness' information in the data during the embedding initialization stage and modeling the entangled feature and sample correlations during the learning process, thus leading to inferior performance. We propose MissGNN, which aims to explicitly leverage the missingness information and such correlations with novel masking schemes. MissGNN first models the data as a bipartite graph and uses a graph neural network to learn node embeddings, where a novel embedding initialization process directly incorporates the missingness information. These embeddings are then optimized through MissGNN's novel feature correlation unit (FCU) and sample correlation unit (SCU), which effectively capture feature and sample correlations for imputation. Experimental results on 8 benchmark datasets against 12 popular and state-of-the-art imputation baselines under three different missingness settings (MCAR, MAR, and MNAR) demonstrate the effectiveness of MissGNN, achieving the best average MAE scores on 7 datasets and the second-best on the remaining one. Our code is publicly available at https://github.com/Alleinx/MissGNN-ICDM2026.**

***Index Terms*—Graph Neural Network, Missing Value Imputation, Feature Correlation, Sample Correlation**

I. Introduction

Missing values in a dataset are a pervasive issue in real-world data analysis. They arise for various reasons, ranging from the limitations of data collection methods to errors during data transmission and storage. To tackle this problem, missing value imputation algorithms have been proposed to preserve all samples and features by imputing missing values with estimated ones based on the observed values in the dataset, so that the dataset can be analyzed completely without discarding incomplete samples or features.

The imputation of missing values usually requires modeling correlations between different features and samples. Feature-wise correlations help predict missing values from other observed features in the same sample, while sample-wise correlations help predict them based on other similar samples. It is thus important to jointly model the feature-wise and sample-wise correlations in the dataset. In addition, the prediction of missing values also largely depends on the 'missingness' of the data, i.e., whether a certain feature value is observed or not in the dataset. Specifically, the missingness information directly determines which observed feature values can be used for imputation. For example, as shown in Figure 1, to impute the same feature value in different samples, we need to leverage different observed features in these samples due to their distinct missingness patterns. Hence, given a target entry to impute, it still remains a challenging problem to automatically select relevant features or samples for imputation under different kinds of data missingness.

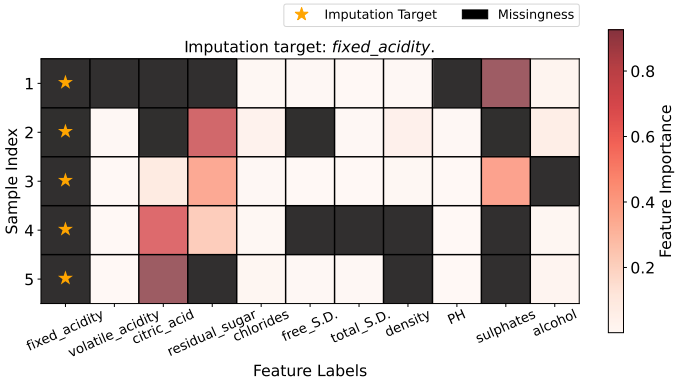


Fig. 1: MissGNN dynamically adjusts the importance of observed features under varying missingness patterns in different samples.

Among existing methods for missing value imputation, traditional ones [1]–[4] extract data correlations with statistical models, which are generally not flexible in handling mixed data types and struggle to scale up to large datasets. In contrast, recent learning-based imputation methods [5]–[9] take advantage of the strong expressiveness and scalability of machine/deep learning algorithms to model data correlations. Other approaches [10], [11] further incorporate observation masks to indicate available entries in samples during imputation, thereby improving imputation accuracy. However, most of them are still built upon the raw tabular data structure as is, which greatly restricts them from jointly modeling the feature-wise and sample-wise correlations.

*: corresponding author

In light of this, graph-based methods [12]–[15] have been proposed to model the raw tabular data as a bipartite graph to take advantage of graph structure in jointly modeling complex dependencies between features and samples [16], [17]. In particular, samples and features serve as two distinct types of nodes and an edge connects a sample node to a feature node if the feature value is observed in that sample. Missing values are then typically predicted by computing the inner product between the learned embeddings of the corresponding sample and feature nodes. However, while this bipartite graph intrinsically captures the data-missingness information, existing methods fail to fully exploit it during the final prediction. Specifically, they learn static, 'global' feature-wise and sample-wise embeddings and apply them uniformly to predict all missing values. Because they do not explicitly incorporate target-specific observation masks into the imputation step, these methods cannot dynamically adjust the correlation weights for a specific target entry based on the distinct missingness patterns across different samples.

In this work, we address these problems by proposing MissGNN, a missingness-guided graph neural network (GNN) for feature- and sample-aware imputation. The key idea behind MissGNN is to explicitly utilize the data-missingness information as model input with our proposed novel masking schemes so that it can automatically leverage the most relevant features and samples for imputation in the presence of different kinds of data missingness. Specifically, MissGNN first builds a bipartite graph from the data. In the embedding initialization for graph representation learning, we not only use the relationships between samples and their associated features but also the missingness information to initialize the embeddings of samples and features jointly and effectively. We then propose our novel feature correlation unit (**FCU**) and sample correlation unit (**SCU**) in MissGNN to explicitly take feature-wise and sample-wise correlations into account for imputation. **FCU** learns the correlations between the target missing feature and observed features within each sample, which are then further updated via a soft mask on the sample missingness information. **SCU** then computes the sample-wise correlations with another soft mask on the missingness information for each pair of samples that have values to impute. We then integrate the output embeddings of **FCU** and **SCU** such that feature-wise and sample-wise correlations are dynamically adjusted based on the missingness information in estimating the missing values. Our contributions can be summarized as follows:

- We propose an effective graph-based imputation method MissGNN for tabular data, which consists of a novel initialization unit that incorporates the missingness patterns, a feature correlation unit (**FCU**), and a sample correlation unit (**SCU**). To the best of our knowledge, MissGNN is the first method that *explicitly* adjusts both feature and sample correlations dynamically based on different imputation targets and missingness patterns.
- The novel **FCU** module in MissGNN effectively learns the correlations between target missing features and observed features within a sample in the imputation process. This allows MissGNN to leverage the most relevant observed features for imputing different missing values.
- The novel **SCU** module in MissGNN leverages information from relevant peer samples and introduces a novel sample correlation measure that is not only accurate but also enables dynamic updates of sample correlations for different imputation targets and missingness patterns.
- We carry out extensive experiments on 8 open datasets. The results show that MissGNN outperforms 12 popular and state-of-the-art methods on 7 of the 8 datasets on average under three different missingness settings, i.e., MCAR, MAR, and MNAR, achieving up to 13.33% improvement in MAE compared to the second-best method.

II. Related Work

A. Traditional Methods

Traditional imputation approaches include joint modeling with expectation maximization (EM) [18], [19], k-nearest neighbors (kNN) [20], [21], and matrix completion [3], [22], [23]. However, joint modeling with EM and matrix completion often lacks the flexibility to handle data with mixed modalities, while kNN faces scalability issues due to its high computational complexity. In contrast, MissGNN is scalable and adaptive to different data distributions.

B. Learning-Based Methods

Iterative imputation frameworks [24]–[28], such as Mice [26] and HyperImpute [24], have been extensively studied. These iterative frameworks apply different imputation methods for each feature and iteratively estimate missing values until convergence. In addition, for deep neural network learners, both generative models [5], [29]–[38] and discriminative models [10], [11], [25] have also been proposed. Among them, ReMasker [10] re-masks a subset of observed values in addition to the existing missing entries and trains an autoencoder to reconstruct the re-masked values from the remaining observations. The trained model is then used to impute the actual missing values, with an observation mask indicating the observed entries for each prediction. Furthermore, several recent studies [39]–[41] propose to adopt pre-trained language models to predict missing values in tabular data, while [8] focuses on accelerating the imputation on tabular data under imputation accuracy guarantees, and [9] proposes to perform imputation and clustering over incomplete data. However, these methods are built upon raw tabular data structures, which may fall short of capturing the complex feature-wise and sample-wise correlations. In contrast, MissGNN is based on the bipartite graph modeling of the data, which is more suitable for learning the data correlations for imputation.

C. Graph Neural Network-Based Methods

GNN-based methods [12]–[15] have been proposed to address the drawbacks mentioned above due to their effectiveness in modeling complex relations between entities. Among them, Grape [12] transforms tabular data into a bipartite

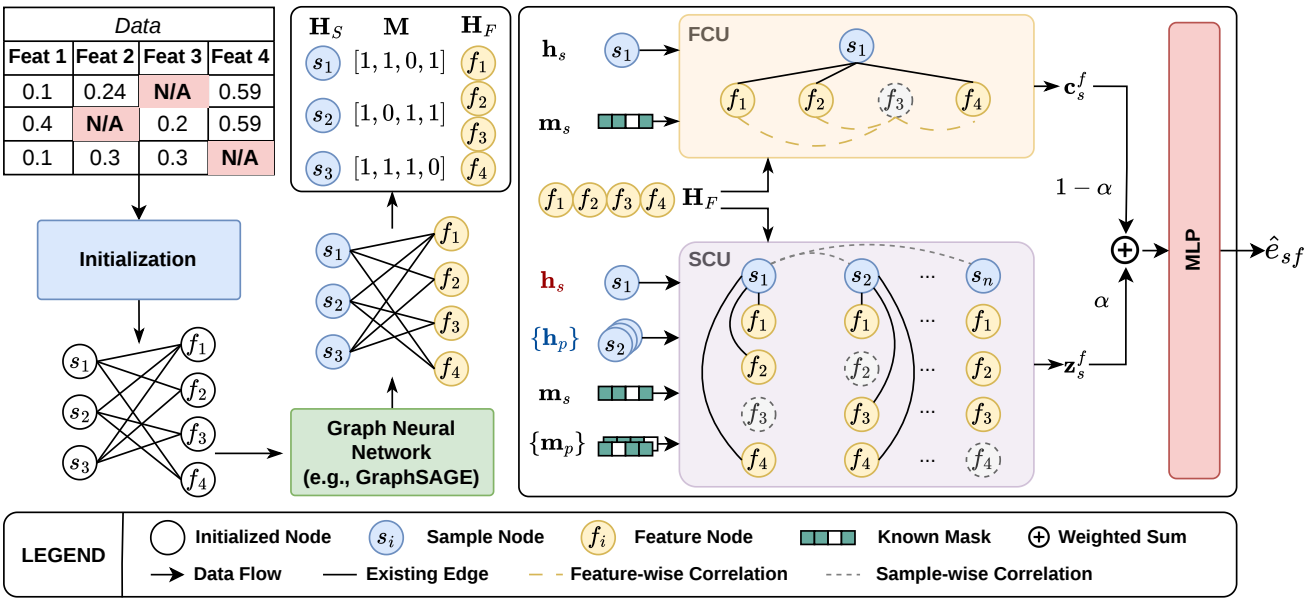


Fig. 2: Overview of the MissGNN model. The tabular data with missing values is first modeled as a bipartite graph with our refined initialization unit, which incorporates the missingness information in node embedding initialization. The graph is then processed with a GNN to update node embeddings. After that, we apply our novel soft masking schemes on these node embeddings to further encode correlation and missingness information in the learning process, using our novel components of feature correlation unit (**FCU**) and sample correlation unit (**SCU**). Eventually, the missing value is predicted with an MLP on the weighted sum of the outputs from **FCU** and **SCU**.

graph where features represent one type of node and samples the other. This transformation allows the imputation task to be framed as a link prediction problem, where the inner product of the learned node embeddings is computed as the predicted values. However, these methods do not explicitly use observation masks to indicate observed features in samples during imputation, which can impair their imputation accuracy. In contrast, MissGNN enables explicit modeling of missingness information through our novel **FCU** and **SCU** units, as well as our novel initialization unit, so that feature-wise and sample-wise correlations can be accurately captured in the imputation process.

III. MissGNN

A. Overview

We here provide an overview of MissGNN to impute the missing value of feature f for a given sample s, as depicted in Figure 2. Initially, the data matrix with missing values is modeled as an undirected bipartite graph, and the missing value is imputed by predicting the edge weight $\hat{e}_{sf}$ of its corresponding missing edge (Section III-B). MissGNN next employs a GNN model, such as GraphSAGE [42], on the bipartite graph to learn the embeddings of samples and features. These embeddings, along with the known masks of the data matrix (used to indicate which feature values are available in each sample), are then input into our novel feature correlation unit (**FCU**) and sample correlation unit (**SCU**), which shall be explained in Section III-C and Section III-D, to obtain feature-wise and sample-wise correlations, respectively. Finally, MissGNN takes the feature-wise and sample-wise correlations into a multi-layer perceptron (MLP) to predict the missing feature value $\hat{e}_{sf}$ (Section III-E). The whole process, including the embedding generation, is trained in an end-to-end manner.

B. Initialization Unit

Let $\mathbf{A} \in \mathbb{R}^{n \times m}$ be an $n \times m$ matrix that consists of n data samples and m features, where $\mathbf{A}_{ij}$ denotes the j-th feature value of the i-th data sample. We introduce an $n \times m$ mask matrix $\mathbf{M} \in \{0,1\}^{n \times m}$ for $\mathbf{A}$ to indicate that the value of $\mathbf{A}_{ij}$ is *observed* when $\mathbf{M}_{ij} = 1$. In other words, the goal of imputation here is to predict the missing feature values $\mathbf{A}_{ij}$ for i and j such that $\mathbf{M}_{ij} = 0$. We define the *masked* data matrix $\mathbf{D}$ to be $\mathbf{D} = \mathbf{A} \odot \mathbf{M}$, where $\odot$ is the Hadamard product, i.e., the element-wise multiplication of two matrices.

As used in recent studies [12], [14], [15], we model the masked data matrix $\mathbf{D}$ as a bipartite graph and tackle the missing value imputation problem as a link prediction task on the bipartite graph. Specifically, $\mathbf{D}$ is modeled as an undirected bipartite graph $\mathcal{G} = (\mathcal{S} \cup \mathcal{F}, \mathcal{E})$, where $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$ is the set of 'sample' nodes and $\mathcal{F} = \{f_1, f_2, \ldots, f_m\}$ is the set of 'feature' nodes. Also, $\mathcal{E}$ is the set of edges that only exist between sample node s and feature node f when $\mathbf{M}_{sf} \neq 0$, and each edge $(s, f) \in \mathcal{E}$ is associated with edge weight e_{sf}, which is given by $e_{sf} = \mathbf{D}_{sf}$. Then, the missing value imputation problem becomes, for any missing entries in $\mathbf{D}$ (where $\mathbf{M}_{sf} = 0$), to predict their corresponding edge weights by developing a learnable mapping $F(\cdot)$, i.e.,

$$\hat{e}_{sf} = F(\mathcal{G}, (s, f) \notin \mathcal{E}). \tag{1}$$

The recent studies [12], [14], [15] that use the bipartite graph modeling initialize all sample node embeddings as all-one vectors and feature node embeddings as one-hot vectors, which have a value 1 in the positions representing their

respective features and 0's elsewhere. However, we observe that such an initialization does not effectively utilize the information from the masked data matrix, which leads to inferior imputation accuracy, as shall be demonstrated in Section IV-D. Thus, in MissGNN, we propose to initialize each sample node embedding based on its associated (initial) feature embeddings instead of initializing them separately. While the feature embeddings are randomly initialized, the sample node embeddings are initialized in a way that reflects the embeddings of the features whose values are available in their corresponding samples.

Let $\mathbf{h}_f^0$ be the initial embedding of feature f, which is a randomly initialized d-dimensional vector, and define $\mathbf{H}_F^0 = [\mathbf{h}_{f_1}^0 \mathbf{h}_{f_2}^0 \dots \mathbf{h}_{f_m}^0] \in \mathbb{R}^{d \times m}$. Also, let $\mathbf{d}_s \in \mathbb{R}^m$ be the s-th column vector of $\mathbf{D}^\top$, which is a vector of the feature values of sample s, and let $\mathbf{m}_s \in \mathbb{R}^m$ be its corresponding mask vector, i.e., $\mathbf{m}_s = \text{col}_s(\mathbf{M}^\top)$, where $\text{col}_s(\cdot)$ denotes the s-th column vector of the matrix. We then initialize the embedding $\mathbf{h}_s^0$ of each sample node s as follows:

$$\mathbf{h}_s^0 = \phi\big(\mathbf{H}_F^0\big[\mathbf{d}_s + \epsilon(\mathbb{1} - \mathbf{m}_s)\big]\big), \tag{2}$$

where $\mathbb{1} \in \mathbb{R}^m$ is an all-one vector, and $\phi(\cdot)$ is an MLP. Note that the term $\mathbf{d}_s + \epsilon(\mathbb{1} - \mathbf{m}_s)$ indicates a vector that consists of observable feature values of s and some small positive values ϵ in the places where the feature values are missing.

C. Feature Correlation Unit

To improve the accuracy of missing value imputation, we aim to fully exploit feature correlations which often appear in the datasets. While the feature correlations are naturally captured by GNNs, we observe that there is still room for improvement. We propose **FCU** as an integral component of MissGNN to fully exploit the feature correlations.

To impute the missing value of feature f for a given sample s, **FCU** begins by computing the feature 'context' vector of sample s in the embedding space that reflects the correlations between the target missing feature f and observed features. Let $\mathbf{h}_f \in \mathbb{R}^d$ be the learned embedding vector of feature f from the GNN, and let $\mathbf{H}_F$ be the $d \times m$ matrix that consists of all the learned feature embedding vectors. We first obtain dot-product similarities between feature f and all the features in the embedding space, i.e., $\mathbf{H}_F^\top \mathbf{h}_f$. We then mask out the similarity values with respect to *non-observed* features in sample s. Here, instead of applying the mask vector $\mathbf{m}_s$ of sample s directly, we use a learnable 'soft' mask vector, denoted by $\mathbf{m}'_s$, which is defined to be $\mathbf{m}'_s = \sigma_1(\mathbf{m}_s) \in \mathbb{R}^m$, where $\sigma_1(\cdot)$ is an MLP with the GELU activation function. In other words, we obtain feature-wise similarities with respect to sample s, denoted by $\mathbf{r}_s^f$, as follows:

$$\mathbf{r}_s^f = \sigma_2\left((\mathbf{H}_F^\top \mathbf{h}_f) \odot \mathbf{m}'_s\right) \in \mathbb{R}^d, \tag{3}$$

where $\sigma_2(\cdot)$ denotes another MLP with the GELU activation function.

FCU next obtains the Hadamard product between the learned embedding vector of sample s, $\mathbf{h}_s$, and the feature-wise similarities with respect to sample s, $\mathbf{r}_s^f$, to learn their joint representations in a multiplicative manner. Specifically, **FCU** obtains the feature context vector of sample s, denoted by $\mathbf{c}_s^f$, as follows:

$$\mathbf{c}_s^f = \sigma_3\left(\mathbf{h}_s \odot \mathbf{r}_s^f\right) \in \mathbb{R}^d, \tag{4}$$

where $\sigma_3(\cdot)$ is also an MLP with the GELU activation function. That is, **FCU** fuses the representation vector of s and the vector that has embedding similarity values between the target feature f and the available features in s through the effective use of the soft mask $\mathbf{m}'_s$. From (3) and (4), the operations of **FCU** can be written as:

$$\begin{aligned}\mathbf{c}_s^f &= \textbf{FCU}(\mathbf{h}_s, \mathbf{m}_s, \mathbf{H}_F) \\ &= \sigma_3\left(\mathbf{h}_s \odot \sigma_2\left((\mathbf{H}_F^\top \mathbf{h}_f) \odot \sigma_1(\mathbf{m}_s)\right)\right).\end{aligned} \tag{5}$$

We empirically show in Figure 3 that FCU can adaptively adjust the weights of observable features based on both the imputation targets and the missingness patterns. Figure 3 describes the scenario where we impute two different missing values within the same sample. Here the missingness is represented by black cells, while each target feature value to impute is marked with a star. In the first row, when imputing the second feature (*volatile_acidity*), the **FCU** identifies the fourth feature (*residual_sugar*) and the last feature (*alcohol*) as the most important features, while assigning low importance to the other observable features. This makes sense as fixed acids such as citric acid are non-volatile and do not easily evaporate. Similarly, in the second row, when imputing the third feature (*citric_acid*), the **FCU** considers the first feature (*fixed_acidity*) to be more important than the other features.

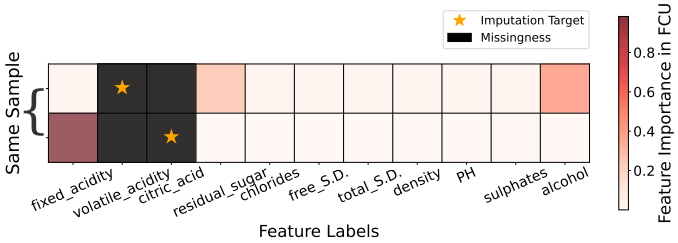


Fig. 3: Feature importance scores calculated from FCU when imputing two missing entries in a sample. FCU dynamically adjusts the importance of observed features within the sample when imputing different feature values.

D. Sample Correlation Unit

To measure similarities between s and other samples, a common approach would be to use the dot product or cosine similarity between their embedding vectors. This approach, however, fails to take into account the observability or availability of each feature in a sample. It also does not capture the fact that different observed features are of different importance to the target feature to impute when it comes to measuring the similarities. We introduce **SCU** as another integral component of MissGNN to compute the sample 'context' vector of sample s by incorporating the embedding vectors of its similar samples as well as different weights of observed features. **SCU** works based on the two novel masking schemes, which shall be explained shortly.

Suppose we are to impute the missing value of feature f for a given sample s. **SCU** aims to leverage the information from the samples that are similar to s. As a first step to this end, we create a subset of samples $\mathcal{P} \subset \mathcal{S}$ that are similar to s. Specifically, we randomly choose and put a sample into $\mathcal{P}$ with probability that is proportional to the cosine similarity between s and the sample. This operation is repeated without replacement until $\mathcal{P}$ reaches a given size.

Mutual Sample Masking: Given a subset of samples $\mathcal{P}$ that include s, we first compute the pairwise similarities between s and other samples in the subset $\mathcal{P}$. While they are computed in a similar way to **FCU**, we only consider the commonly observed features (or the common ones that have feature values) in both s and its peer $p \in \mathcal{P} \setminus \{s\}$, to calculate their pairwise similarity in the sense that the missing value of feature f is inferred. Specifically, we compute the pairwise similarity between s and $p \in \mathcal{P} \setminus \{s\}$, which is denoted by $\text{sim}(s, p \mid f)$, as follows:

$$\text{sim}(s, p \mid f) = \textbf{FCU}(\mathbf{h}_s, \mathbf{m}_p, \mathbf{H}_F) \cdot \textbf{FCU}(\mathbf{h}_p, \mathbf{m}_s, \mathbf{H}_F), \quad (6)$$

where $\mathbf{h}_s$ and $\mathbf{h}_p$ are the learned embedding vectors of samples s and p from the GNN, respectively, and $\mathbf{m}_s$ and $\mathbf{m}_p$ are their respective mask vectors.

Irrelevant Feature Masking: After we obtain the pairwise similarities between s and other samples in $\mathcal{P}$, it would be natural to consider a weighted sum of their corresponding embedding vectors, i.e., $\sum_{p \in \mathcal{P} \setminus \{s\}} \text{sim}(s, p \mid f)\ \mathbf{h}_p$, in imputing the value of the target feature f. However, we observe that $\mathbf{h}_p$ contains the information from the features whose values are available in p as well as possibly other features, as it is learned via the so-called neighborhood aggregation mechanism that is central to GNNs, but some of the features may be irrelevant in inferring the value of feature f. Thus, instead of using $\{\mathbf{h}_p\}$ directly, we introduce a d-dimensional mask vector $\mathbf{r}_p^f$ for $\mathbf{h}_p$, which is to mask out potentially irrelevant feature information in $\mathbf{h}_p$, when it comes to imputing the value of feature f. Specifically, it is defined by

$$\mathbf{r}_p^f = \sigma_4\left([\mathbf{m}_p; \overline{\mathbf{m}}_f]\right) \in \mathbb{R}^d, \quad (7)$$

where $\overline{\mathbf{m}}_f$ is an m-dimensional one-hot vector that has a value 1 in the place of feature f and 0's elsewhere, $[\cdot\,;\cdot]$ denotes the vector concatenation operation, and $\sigma_4(\cdot)$ is an MLP with the GELU activation function. Note that the rationale behind the design of $\mathbf{r}_p^f$ is to embed the information on the features whose values are present in p as well as the information on the target feature f to impute.

The mask $\mathbf{r}_p^f$ is then applied to $\mathbf{h}_p$ to obtain the masked embedding vector of p as follows:

$$\phi_p(\mathbf{h}_p, \mathbf{r}_p^f) = \sigma_5\left(\mathbf{h}_p \odot \mathbf{r}_p^f\right) \in \mathbb{R}^d, \quad (8)$$

where $\sigma_5(\cdot)$ is also an MLP with the GELU activation function. Once we have the masked embedding vectors of samples (excluding s) in $\mathcal{P}$, we finally compute the sample context vector of sample s, denoted by $\mathbf{z}_s^f$, which is a weighted sum of the masked embedding vectors with weights being the pairwise similarity values, i.e.,

$$\mathbf{z}_s^f = \sigma_6\Big(\sum_{p \in \mathcal{P} \setminus \{s\}} \text{sim}(s, p \mid f)\ \phi_p(\mathbf{h}_p, \mathbf{r}_p^f)\Big) \in \mathbb{R}^d, \quad (9)$$

where $\sigma_6(\cdot)$ is again an MLP with the GELU activation function.

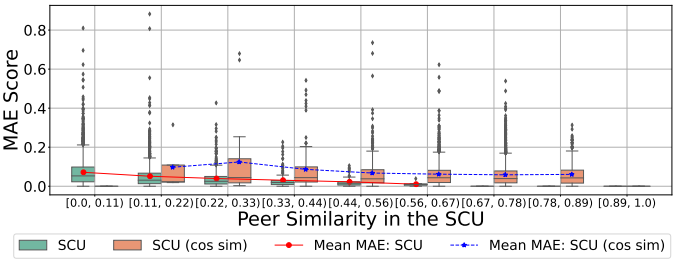


Fig. 4: MAE scores and peer similarity scores measured by two methods. Data are missing under the MCAR setting with 30% missingness.

We empirically show in Figure 4 that our **SCU** is more effective than cosine similarity in measuring sample correlations, and leads to improved imputation accuracy. We first introduce a variant of **SCU**, denoted as "**SCU (cos sim)**", in which we change Equation (6) with cosine similarity while keeping the remaining computations in MissGNN unchanged. The updated equation is now defined as:

$$\text{sim}(s, p \mid f) = \cos(\mathbf{h_s},\ \mathbf{h_p}). \quad (10)$$

We then use **SCU** and **SCU (cos sim)** to compute the peer similarity scores separately for imputation. Intuitively, if the calculated similarity scores well capture the peer correlations, the higher the similarity scores, the lower the MAE.

As shown in Figure 4, MAE of MissGNN with **SCU** decreases with higher similarity scores and consistently achieves lower mean MAE scores than **SCU (cos sim)**, demonstrating the effectiveness of our novel similarity measure. In contrast, MAE of **SCU (cos sim)** first increases and then decreases, showing that the correlations may be incorrectly calculated to some extent. Further evaluations in Table VII also show that **SCU** consistently outperforms **SCU (cos sim)** across various datasets. All these results validate the effectiveness of **SCU** in measuring peer similarity more accurately.

E. Imputation

For a given sample s, to impute the missing value of feature f, MissGNN obtains its feature context vector $\mathbf{c}_s^f$ and sample context vector $\mathbf{z}_s^f$ through **FCU** and **SCU**, respectively, which are then used for imputation. Specifically, it is done by predicting the corresponding edge weight $\hat{e}_{sf}$ as follows:

$$\hat{e}_{sf} = \phi_\alpha\left((1-\alpha)\mathbf{c}_s^f + \alpha \mathbf{z}_s^f\right), \quad (11)$$

where $\phi_\alpha(\cdot)$ denotes an MLP with a non-linear activation function (i.e., ReLU for continuous values and softmax for discrete ones), and α is a learnable scalar parameter. This scalar parameter α is introduced to strike a balance between leveraging feature-wise correlation and sample-wise correlation.

Algorithm 1 Forward computation of MissGNN to impute the value of feature f for sample s.

Input: Bipartite graph $\mathcal{G}$, initial feature node embeddings $\mathbf{H}_F^0$, GNN model (e.g., GraphSAGE) $\mathbf{GNN}(\cdot)$, known mask matrix $\mathbf{M}$, and a subset of samples $\mathcal{P} \subset \mathcal{S}$.
Output: Predicted missing feature value $\hat{e}_{sf}$.
Obtain initial sample-node embeddings $\mathbf{H}_S^0$, where the embedding $\mathbf{H}_s^0$ of sample $s \in \mathcal{S}$ is computed according to Equation (2).
$\mathbf{H}_S, \mathbf{H}_F = \mathbf{GNN}(\mathbf{H}_S^0, \mathbf{H}_F^0, \mathcal{G})$.
$\mathbf{c}_s^f = \mathbf{FCU}(\mathbf{h}_s, \mathbf{m}_s, \mathbf{H}_F)$.
$\mathbf{z}_s^f = \mathbf{SCU}(\mathbf{H}_\mathcal{P}, \mathbf{M}_\mathcal{P}, \mathbf{H}_F)$.
Predict the missing feature value $\hat{e}_{sf}$ using Equation (11).

This is necessary because the quality of $\mathbf{z}_s^f$ relies on the quality of the samples chosen in $\mathcal{P}$, so overly relying on $\mathbf{z}_s^f$ would backfire if their quality is not as desired. To address this problem, instead of employing a fixed weight α, we make α learnable and adaptive in determining the weights for $\mathbf{c}_s^f$ and $\mathbf{z}_s^f$. The scalar parameter α is learned based on the similarity values between s and its peer samples $p \in \mathcal{P} \setminus \{s\}$ as follows:

$$\alpha = \phi_\gamma\Big(\Big\|_{p \in \mathcal{P} \setminus \{s\}} \text{sim}\,(s, p \mid f) \Big), \tag{12}$$

where $\|$ represents the concatenation operation, and $\phi_\gamma(\cdot)$ is an MLP with the activation function $\gamma(x) = 1 - 1 \,/\, e^{|x|}$.

To learn network parameters, we use cross-entropy loss and mean square error loss for imputing discrete and continuous feature values, respectively. The overall procedure of MissGNN is summarized in Algorithm 1. We analyse the time complexities of **initialization**, **FCU**, **SCU**, and **prediction** in Algorithm 1. Let n, m, and d denote the numbers of samples, features, and the embedding dimension, respectively. Let $k = |\mathcal{P}| - 1 \geq 1$. Each MLP has L hidden layers of width d. The **initialization** complexity in Eq. (2) is $O\big(n(md + Ld^2)\big)$, with $O(md)$ from vector and matrix-vector operations, and $O(Ld^2)$ from $\phi(\cdot)$. The **FCU** complexity in Eq. (5) is $O(md + Ld^2)$, where $\mathbf{H}_F^\top \mathbf{h}_f$ costs $O(md)$, σ_1, σ_2 cost $O(md + Ld^2)$, and σ_3 costs $O(Ld^2)$. The **SCU** complexity in Eq. (9) is $O\big(k(md + Ld^2)\big)$, where every peer requires $\text{sim}(s, p \mid f)$ and $\phi_p(\mathbf{h}_p, \mathbf{r}_p^f)$ operations, with each costing $O(md + Ld^2)$. Finally, **prediction** in Eq. (11) takes $O(kd + Ld^2)$. Since $\mathbf{H}_s^0$, $\mathbf{H}_S$, and $\mathbf{H}_F$ can be computed once and reused across different missing entries, the per-entry complexity after initialization and GNN encoding is $O\big(k(md + Ld^2)\big)$, accounting for **FCU**, **SCU**, and **prediction**.

IV. Experiments

A. Experiment Setup

1) Datasets: We conduct experiments on 8 open datasets, with statistics summarized in Table I. These real-world datasets consist of mixed data types with both continuous and discrete values and cover different domains, including civil engineering, physics, and chemistry. We follow the preprocessing steps adopted in Grape [12] to scale feature values to [0, 1] with a MinMax scaler [43].

2) Missingness Settings: As used in prior studies [24], [25], we apply three masking generation schemes, namely missing completely at random (MCAR), missing at random (MAR), and missing not at random (MNAR). We adopt the same procedure outlined in [24], [25] to generate missing values under different settings.

- **MCAR**: For the MCAR setting, given a dataset containing n samples and m features, we generate an $n \times m$ matrix as a mask. Each entry value of the matrix is sampled from a uniform distribution. Entries with values no greater than the missingness ratio are treated as missing, i.e., masked out, and the remaining entries remain observable.
- **MAR**: For the MAR setting, we randomly select a subset of features to remain fully observed and introduce missing values only into the remaining features. Entries in these features are masked according to probabilities generated by a logistic model with random weights, using the fully observed features as input. The desired missingness ratio is achieved by adjusting the bias term.
- **MNAR**: For the MNAR setting, we first apply the MAR mechanism. Once its output is generated, we then mask additional feature values in the set of fully observed features using the MCAR mechanism.

3) Baseline models: MissGNN is compared against popular and state-of-the-art imputation methods, including statistical methods, deep generative methods, and graph-based methods listed as follows: **Mean**, **kNN** [21], **Mice** [26], **Svd** [4], **Spectral** [3], **Miwae** [30], **Miracle** [25], **Gain** [29], **Grape** [12], **HyperImpute** [24], **IGRM** [14], and **ReMasker** [10]. We follow the official implementations of all the baseline models.

4) Implementation Details: Parameters of MissGNN are updated by the Adam optimizer with a learning rate of 0.001 for 40,000 epochs. For graph representation learning, we use a three-layer E-GraphSAGE model, which is a variant of GraphSAGE [42] that not only leverages node embeddings but also edge embeddings during the neighborhood aggregation mechanism, as similarly used in Grape [12]. We employ mean-pooling as the aggregation function and use ReLU as the activation function for the GNN layers. We set the embedding dimension d to 128. We randomly drop 50% of observable edges during training to improve the model's generalization ability. For each experiment, we conduct five runs with different random seeds and report the average results.

B. Overall Performance

We first compare the feature imputation performance of MissGNN with 12 popular and state-of-the-art imputation methods. Table II summarizes the mean absolute error (MAE) under the MCAR, MAR, and MNAR settings with 30% missingness. Among the 24 sets of evaluations, MissGNN achieves 20 best and 4 second-best performance, substantially outperforming all baseline methods. This is because the explicit modeling of the missingness information through our novel soft masking schemes in **FCU** and **SCU** as well as the initialization unit is effective in handling different patterns of missing values in the input data. Furthermore, MissGNN is designed in a

TABLE I: Overview of datasets, which contains continuous features and discrete features. We use the abbreviations (letters in boldface) to represent the datasets in the evaluation.

Statistics \ Dataset	**YA**cht	**ST**eel	**GE**rman	**CO**ncrete	**HO**using	**IO**nosphere	**PO**wer	**CM**c
# Samples	308	1941	1000	1030	768	351	9568	1473
# Continuous Features	6	33	13	12	8	34	4	8
# Discrete Features	0	0	7	1	0	0	0	1

TABLE II: Imputation accuracy in MAE under the MCAR, MAR, and MNAR settings. MAE scores are enlarged by 10 times. The best and the second best performers are marked in **bold** and underlined, respectively. Dataset names are summarized in Table I.

	Dataset	**Mean**	**Svd**	**Spectral**	**Mice**	**kNN**	**Gain**	**Miwae**	**Grape**	**Miracle**	**HyperImpute**	**IGRM**	**ReMasker**	**MissGNN**
MCAR	**YA**	2.09	2.46	2.64	1.68	1.67	2.26	4.68	1.46	3.84	1.76	**1.02**	1.77	1.33
	ST	1.80	1.37	1.10	0.95	0.78	1.03	1.70	0.45	0.95	0.72	0.47	0.53	**0.39**
	GE	2.52	2.60	2.45	2.26	2.34	2.27	2.59	2.01	3.67	2.05	2.01	2.03	**1.87**
	CO	1.79	1.94	1.98	1.34	1.16	1.67	1.81	0.75	1.71	0.84	0.76	1.08	**0.71**
	HO	1.85	1.53	1.46	1.16	0.95	1.23	3.81	0.64	3.12	0.82	0.64	1.00	**0.58**
	IO	2.01	1.26	1.38	1.37	1.07	1.50	5.14	1.08	5.56	1.18	1.13	1.70	**1.01**
	PO	1.68	2.33	2.13	1.16	1.38	1.31	1.74	1.00	1.13	1.06	1.01	1.18	**0.98**
	CM	2.35	2.52	2.96	2.06	2.32	2.33	2.37	1.87	2.01	1.91	1.93	1.88	**1.81**
MAR	**YA**	2.20	2.64	3.06	1.79	1.69	2.07	3.47	1.20	3.75	2.06	1.34	1.83	**1.09**
	ST	1.79	1.58	1.32	0.91	0.77	1.25	1.48	0.45	0.84	0.48	0.47	0.74	**0.42**
	GE	2.53	2.65	2.72	2.26	2.43	2.21	2.59	2.08	3.07	2.25	2.03	2.38	**1.91**
	CO	1.79	2.32	2.12	1.27	0.89	1.87	1.87	0.77	1.94	1.30	**0.67**	1.05	0.75
	HO	2.02	1.71	1.84	1.22	0.89	0.92	3.79	0.66	2.24	1.05	0.64	1.36	**0.60**
	IO	2.02	1.40	1.50	1.43	1.09	1.71	6.10	1.17	5.28	1.30	0.92	1.74	**1.07**
	PO	1.81	2.37	3.37	1.16	1.11	1.42	2.07	1.06	**0.86**	1.07	**0.86**	**0.86**	1.01
	CM	2.41	2.60	3.71	2.02	2.11	2.18	2.08	1.83	2.21	2.02	1.86	2.18	**1.74**
MNAR	**YA**	2.18	2.61	2.75	1.91	1.92	2.34	3.77	1.23	3.85	1.95	**0.86**	1.82	1.15
	ST	1.75	1.58	1.24	0.91	0.79	1.27	1.56	0.42	0.89	0.69	0.48	0.58	**0.39**
	GE	2.49	2.67	2.55	2.25	2.46	2.37	2.60	2.04	3.27	2.36	2.02	1.97	**1.88**
	CO	1.80	2.24	1.86	1.37	1.15	1.80	1.86	0.73	1.87	0.88	0.76	1.15	**0.68**
	HO	1.95	1.58	1.60	1.22	0.95	1.08	3.80	0.61	2.51	0.85	0.63	1.01	**0.54**
	IO	2.04	1.36	1.46	1.44	1.12	1.55	5.66	1.15	5.39	1.28	**1.06**	1.74	**1.06**
	PO	1.76	2.27	3.14	1.08	1.65	1.22	1.93	1.02	1.06	1.09	1.01	1.13	**1.00**
	CM	2.37	2.54	2.95	2.36	2.23	2.13	2.15	1.82	2.06	1.91	1.92	2.00	**1.71**

TABLE III: MAE under the **MCAR** setting across different levels of missingness.

	Dataset	**Mean**	**Svd**	**Spectral**	**Mice**	**kNN**	**Gain**	**Miwae**	**Grape**	**Miracle**	**HyperImpute**	**IGRM**	**ReMasker**	**MissGNN**
Miss 10%	**YA**	2.22	1.92	2.24	1.38	1.40	2.30	4.57	1.00	3.87	1.50	**0.67**	1.27	0.96
	CO	1.81	2.04	1.84	0.97	0.58	1.62	1.85	0.45	1.63	0.51	0.57	0.93	**0.41**
	HO	1.84	1.69	1.28	0.98	0.74	1.16	3.78	0.49	3.07	0.75	0.57	1.16	**0.45**
	PO	1.68	2.26	1.77	1.01	1.26	1.22	1.74	0.85	0.99	0.84	0.85	1.08	**0.83**
Miss 30%	**YA**	2.09	2.46	2.64	1.68	1.67	2.26	4.68	1.46	3.84	1.76	**1.02**	1.77	1.33
	CO	1.79	1.94	1.98	1.34	1.16	1.67	1.81	0.75	1.71	0.84	0.76	1.08	**0.71**
	HO	1.85	1.53	1.46	1.16	0.95	1.23	3.81	0.64	3.12	0.82	0.64	1.00	**0.58**
	PO	1.68	2.33	2.13	1.16	1.38	1.31	1.74	1.00	1.13	1.06	1.01	1.18	**0.98**
Miss 50%	**YA**	2.12	3.00	3.17	1.99	2.08	2.33	4.57	1.89	3.84	2.07	**1.68**	2.31	1.74
	CO	1.81	2.19	2.31	1.59	1.40	2.20	1.85	1.24	1.80	1.33	**1.06**	1.49	1.19
	HO	1.84	1.88	1.76	1.33	1.37	1.43	3.79	0.83	3.07	1.06	0.80	0.93	**0.79**
	PO	1.67	2.92	2.98	1.32	1.80	1.36	1.73	**1.19**	1.29	1.29	1.20	1.33	**1.19**
Miss 70%	**YA**	2.16	3.78	4.17	2.21	2.62	3.07	4.56	2.14	3.88	2.49	**2.03**	3.00	2.14
	CO	1.81	2.53	2.75	1.72	1.60	2.84	1.84	1.64	2.23	1.71	**1.39**	2.06	1.56
	HO	1.83	2.58	2.59	1.54	1.43	3.09	3.78	1.12	3.05	1.12	1.10	1.08	**1.08**
	PO	1.67	3.88	4.33	1.46	2.73	1.51	1.72	**1.37**	1.54	1.54	1.39	1.69	**1.37**

way that is agnostic to the underlying missingness patterns. In other words, MissGNN does not use any prior information on the missingness patterns but leverages where missing values appear and how they are related to observable features. Since feature and sample correlations exist regardless of the cause of missingness, MissGNN is naturally adaptive and robust across different missingness settings.

C. Different Levels of Missingness

In practice, datasets may possess different missing ratios. To validate the model's robustness under such circumstances, we evaluate the performance of MissGNN and other baseline models with varying missing ratios, i.e., 10%, 30%, 50%, and 70%. Due to the space limit, we consider four representative datasets from here onward. As shown in Table III, MissGNN consistently outperforms most baseline models across different levels of missingness. This is attributed to the dynamic adjustment of feature and sample correlations by **FCU** and **SCU** in MissGNN. In particular, as previously demonstrated in Figure 1, **FCU** can dynamically adjust feature importance under varying missingness patterns. This allows MissGNN to leverage the most relevant observed features for imputing missing values

TABLE IV: MAE with different ablation architectures and sampling strategies.

	YA	CO	HO	PO
Grape	1.46	0.75	0.64	1.00
Architecture				
Init Only	1.41	0.73	0.62	0.99
Init+**FCU**	1.35	0.72	0.61	0.99
Init+**SCU**	1.37	0.73	0.63	1.02
Sampling Strategy				
MissGNN-Uniform	1.34	0.73	0.61	0.99
MissGNN	**1.33**	**0.71**	**0.58**	**0.98**

and achieving good imputation accuracy, even when the missing ratio is high. In addition, **SCU**, which incorporates **FCU** in its sample correlation measure, can also dynamically update sample correlations and effectively select relevant samples for imputation of different targets. At missing ratios of 50% and 70%, IGRM becomes competitive because the bipartite graph contains fewer edges and becomes sparser, which can hinder the learning process. IGRM mitigates this sparsity issue by introducing additional edges between sample nodes, resulting in better performance on the YA and CO datasets. Nevertheless, MissGNN achieves the second-best performance on these two datasets and the best performance on the HO and PO datasets. Overall, MissGNN is robust to various missing ratios.

D. Ablation Study

To study the effectiveness of three integral components of MissGNN, we consider three variants of MissGNN, each with a subset of the components, namely initialization only (Init Only), initialization + **FCU** (Init + **FCU**), and initialization + **SCU** (Init + **SCU**). As shown in Table IV, the three variants derived from MissGNN achieve lower MAE values than the baseline model Grape for all datasets, demonstrating the effectiveness of our novel components in MissGNN.

Specifically, for initialization only, the key difference between MissGNN and Grape lies in our refined initialization process to explicitly leverage missingness information in node embeddings. The reduced MAE values observed by the Init Only variant demonstrate that our proposed initialization process is more effective in utilizing information between samples and their associated features, including missing ones, as compared to the basic initialization used in Grape. In addition, we observe that when **FCU** or **SCU** is incorporated, MAE values are further reduced for most datasets. This validates that explicitly modeling of missingness information through our novel masking schemes in **FCU** and **SCU** indeed improves imputation accuracy. When all the three components are combined together as in MissGNN, they work synergistically to lower MAE values, validating the efficacy of incorporating the missingness information when capturing sample-wise and feature-wise correlations for missing data imputation.

We also evaluate the effectiveness of the sampling strategy used in **SCU**. While **SCU** uses a sampling strategy based on pairwise cosine similarities to construct a subset of samples $\mathcal{P}$, the simplest sampling strategy to build $\mathcal{P}$ would be to choose samples uniformly at random without replacement (MissGNN-Uniform). Intuitively, this approach cannot identify similar peer samples accurately and thus would lead to inferior performance. As shown in Table IV, it is indeed the case. Notably, even with this naive uniform sampling strategy, MissGNN-Uniform still outperforms the imputation baseline Grape.

TABLE V: Inference time (in seconds) for imputing all missing feature values using different methods. (C) represents CPU running time and (G) indicates GPU running time.

Model	YA	CO	HO	PO
Mean (C)	0.0006	0.0013	0.0018	0.0020
Svd (C)	0.02	0.03	0.04	0.11
Spectral (C)	0.01	0.05	0.05	0.20
Mice (C)	0.03	0.04	0.07	0.05
kNN (C)	0.03	0.27	0.10	17.92
Gain (C)	3.25	3.21	4.24	3.21
Miwae (C)	7.63	23.44	12.13	206.09
Miracle (C)	3.94	8.65	6.23	32.77
HyperImpute (C)	21.68	36.00	131.83	22.61
IGRM (C)	0.01	0.07	0.02	3.31
ReMasker(C)	1.79	5.64	3.05	56.36
Grape (C)	0.05	0.12	0.10	0.63
Grape (G)	0.02	0.02	0.02	0.05
MissGNN (C)	0.05	0.18	0.14	0.78
MissGNN (G)	0.02	0.04	0.04	0.12

TABLE VI: Training time per epoch (in seconds).

Model	YA	CO	HO	PO
Init	0.02	0.02	0.02	0.03
Init + **FCU**	0.02	0.02	0.02	0.04
Init + **SCU**	0.03	0.03	0.03	0.12
MissGNN	0.03	0.04	0.04	0.14

E. Running Time Analysis

In this experiment, we investigate the computational efficiency of MissGNN during training and inference. We first present an inference time comparison in Table V. The results show that our method is both accurate and time-efficient. For example, for inference with GPU, the time taken to impute *all* the missing values for any dataset we tested is less than one second under the setting of MCAR with 30% missingness.

MissGNN is also efficient during training. Table VI reports the GPU runtime of MissGNN during training. As shown, MissGNN demonstrates high efficiency, requiring less than one second to complete the forward and backward passes for a single epoch across all the examined datasets.

*F. **SCU** is More Effective than Cosine Similarity in Measuring Sample Correlations*

Recall that in Section III-D, we show that our **SCU** is more effective than its counterpart with cosine similarity, **SCU (cos sim)**, in measuring sample correlations, thereby leading to improved imputation accuracy. We here quantify the improvements of **SCU** over **SCU (cos sim)**. MAE scores of MissGNN with **SCU** and **SCU (cos sim)** are summarized in Table VII. The results show that our **SCU** consistently outperforms **SCU (cos sim)**, with up to 5.33% improvement

TABLE VII: MAE with **SCU (cos sim)** and **SCU**. Imp. (%) denotes the percentage of improvement relative to **SCU (cos sim)**.

	YA	CO	HO	PO
SCU (cos sim)	1.37	0.75	0.60	1.00
SCU	1.33	0.71	0.58	0.98
Imp. (%)	2.92%	5.33%	3.33%	2.0%

TABLE VIII: MAE for label prediction.

	YA	CO	HO	PO
Mean	9.08	10.50	4.44	6.31
Svd	9.12	10.90	4.26	6.93
Spectral	8.98	10.50	4.33	6.67
Mice	8.95	10.20	3.99	4.99
kNN	8.91	9.95	4.17	5.68
Gain	9.82	10.60	4.20	5.08
Miwae	9.40	10.60	5.43	5.50
Grape	8.96	9.17	3.66	4.83
Miracle	9.70	10.50	5.01	4.93
HyperImpute	9.58	9.94	3.87	4.69
IGRM	9.36	10.36	4.13	5.83
ReMasker	9.73	10.12	3.83	7.04
MissGNN	**8.52**	**8.97**	**3.51**	**4.63**

on the CO dataset. These findings confirm the proposed **SCU** is more effective in measuring sample correlations and leads to improved imputation accuracy.

G. Downstream Task Performance

We further evaluate the quality of imputed values from different imputation methods by performing a downstream label prediction task. Specifically, as each sample in the four examined datasets contains a continuous label, the task is to predict the label using the feature values. Starting with an input data matrix with 30% missingness, we first impute the data using the corresponding imputation methods, and then perform linear regression on the completed data matrix to predict labels. We use 70% of the samples to learn the regression coefficients and evaluate the trained model on the remaining 30%. As shown in Table VIII, MissGNN consistently achieves best performance across examined datasets, indicating its effectiveness in missing value imputation.

H. Parameter Sensitivity Analysis

In this experiment, we evaluate the robustness of MissGNN under various hyperparameter settings. We first evaluate the impact of peer size ($|\mathcal{P}|-1$) in **SCU** on the imputation performance of MissGNN. Intuitively, a proper peer size ($|\mathcal{P}|$–1) should balance high-similarity peers and potential peers that serve for regularization and generalization purposes. In general, the trend across different datasets shows that a too-small peer size may only include high-similarity peers, while a too-large peer size may include too many noisy nodes and incur higher computational overhead. As shown in Table IX, the variation in performance is small, indicating that MissGNN is relatively robust to this parameter. We recommend a peer size of 5–10 for practical use.

We next evaluate whether a non-zero value of ϵ in the initialization unit of MissGNN indeed leads to an improvement in imputation accuracy. As shown in Table IX, for YA and HO datasets, the introduction of a non-zero value of ϵ results in lower MAE scores. Another insight that we have from Table IX is that ϵ should not be set too large, as a large value of ϵ might impose incorrect weights to the features with missing values. We observe that it is an overall good choice to set ϵ to 1×10^{-5} or 1×10^{-4}.

TABLE IX: MAE for varying peer-sample size ($|\mathcal{P}|-1$) and different values of ϵ.

	YA	CO	HO	PO
Peer $= 1$	1.34	0.73	0.61	**0.98**
Peer $= 2$	1.35	0.72	**0.58**	1.00
Peer $= 5$	**1.33**	**0.71**	**0.58**	**0.98**
Peer $= 10$	**1.33**	**0.71**	0.60	1.00
Peer $= 15$	1.34	0.72	0.60	**0.98**
Peer $= 20$	1.34	0.72	0.60	1.00
$\epsilon = 0$	1.34	**0.71**	0.60	**0.98**
$\epsilon = 10^{-5}$	**1.31**	**0.71**	**0.58**	1.00
$\epsilon = 10^{-4}$	1.33	**0.71**	**0.58**	**0.98**
$\epsilon = 10^{-3}$	1.33	0.72	0.60	**0.98**

TABLE X: MAE for different GNN variants.

	YA	CO	HO	PO
MissGNN w/ different GNN variants:				
E-GraphSAGE	**1.33**	**0.71**	**0.58**	**0.98**
GCN	2.04	1.76	1.62	1.66
GAT	1.56	1.01	0.78	1.02
GraphSAGE	1.48	1.02	0.78	1.05
Grape w/ different GNN variants:				
E-GraphSAGE	**1.46**	**0.75**	**0.64**	**1.00**
GCN	2.04	2.24	2.90	1.73
GAT	2.02	1.90	2.13	1.67
GraphSAGE	2.02	1.81	1.74	1.66

We finally conduct experiments using different GNN variants such as E-GraphSAGE, GraphSAGE, GAT, and GCN to evaluate their impact on the imputation accuracy. Note that E-GraphSAGE is an edge-aware variant of GraphSAGE [42] that incorporates edge embeddings in message aggregation. Since MissGNN can be used with a different GNN variant, we compare the performance of MissGNN against Grape, which can also be used with a different GNN. The results in Table X indicate that different GNN variants may introduce varying levels of imputation accuracy, but MissGNN consistently outperforms its Grape counterpart for different GNN variants, demonstrating its effectiveness.

V. Conclusion

In this paper, we highlighted the importance of missingness information and presented MissGNN, a missingness-guided representation learning method for missing value imputation. MissGNN improved the embedding initialization process by considering the relationships between samples and their associated features (including missing ones). In addition, for more effective representation learning, we introduced two novel components in MissGNN, i.e., **FCU** and **SCU**, which explicitly model the missingness information with our soft masking schemes to better capture data correlations for imputation.

Extensive experimental results on 8 open datasets demonstrate the effectiveness of MissGNN, achieving overall superior performance compared to 12 popular and state-of-the-art methods under three different settings of missing value patterns.

ACKNOWLEDGMENT

This work was supported in part by the Research Grants Council Collaborative Research Fund under grant number (C1045-23G), the RGC-General Research Fund under grant number (16201625) of Hong Kong, and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) under Grant RS-2024-00349885. This work was also supported in part by the Guangdong and Hong Kong Universities "1+1+1" Joint Research Collaboration Scheme under grant number (2025A0505000004 and 2025A0505000012).

REFERENCES

[1] L. F. Burgette and J. P. Reiter, "Multiple imputation for missing data via sequential regression trees," *American journal of epidemiology*, 2010.

[2] J. Honaker, G. King, and M. Blackwell, "Amelia ii: A program for missing data," *Journal of statistical software*, 2011.

[3] R. Mazumder, T. Hastie, and R. Tibshirani, "Spectral regularization algorithms for learning large incomplete matrices," *The Journal of Machine Learning Research*, 2010.

[4] T. Hastie, R. Mazumder, J. D. Lee, and R. Zadeh, "Matrix completion and low-rank svd via fast alternating least squares," *The Journal of Machine Learning Research*, 2015.

[5] S. Zheng and N. Charoenphakdee, "Diffusion models for missing value imputation in tabular data," in *NeurIPS Table Representation Learning (TRL) Workshop*, 2022.

[6] Y. Tashiro, J. Song, Y. Song, and S. Ermon, "CSDI: conditional score-based diffusion models for probabilistic time series imputation," in *Advances in Neural Information Processing Systems*, 2021.

[7] B. Muzellec, J. Josse, C. Boyer, and M. Cuturi, "Missing data imputation using optimal transport," in *Proceedings of the 37th International Conference on Machine Learning*, 2020.

[8] X. Miao, Y. Wu, L. Chen, Y. Gao, J. Wang, and J. Yin, "Efficient and effective data imputation with influence functions," *Proceedings of the VLDB Endowment*, 2021.

[9] Y. Sun, J. Zhu, X. Xu, X. Xu, Y. Sun, S. Song, X. Li, and X. Yuan, "Win-win: On simultaneous clustering and imputing over incomplete data," *Proceedings of the VLDB Endowment*, 2024.

[10] T. Du, L. Melis, and T. Wang, "Remasker: Imputing tabular data with masked autoencoding," in *The Twelfth International Conference on Learning Representations*, 2024.

[11] R. Wu, A. Zhang, I. Ilyas, and T. Rekatsinas, "Attention-based learning for missing data imputation in holoclean," *Proceedings of Machine Learning and Systems*, 2020.

[12] J. You, X. Ma, D. Y. Ding, M. J. Kochenderfer, and J. Leskovec, "Handling missing data with graph representation learning," in *Advances in Neural Information Processing Systems*, 2020.

[13] I. Spinelli, S. Scardapane, and A. Uncini, "Missing data imputation with adversarially-trained graph convolutional networks," *Neural Networks*, 2020.

[14] J. Zhong, N. Gui, and W. Ye, "Data imputation with iterative graph reconstruction," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[15] P.-C. Liu and C.-T. Li, "Self-supervised bipartite graph neural networks with missing value imputation for small tabular data predictions," *ACM Transactions on Intelligent Systems and Technology*, 2026.

[16] Z. Yu, J. Wu, Z. Wu, S. Zhong, W. Su, C.-H. Lee, and W. Zhuo, "Tami: Taming heterogeneity in temporal interactions for temporal graph link prediction," in *Advances in Neural Information Processing Systems*, 2025.

[17] X. Huang, W. Zhuo, M. P. Vuong, S. Li, J. Kim, B. Rees, and C.-H. Lee, "Sdt-gnn: Streaming-based distributed training framework for graph neural networks," in *IEEE International Conference on Big Data*, 2025.

[18] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the em algorithm," *Journal of the royal statistical society: series B (methodological)*, 1977.

[19] Z. Ghahramani and M. Jordan, "Supervised learning from incomplete data via an em approach," in *Advances in neural information processing systems*, 1993.

[20] P. J. García-Laencina, J.-L. Sancho-Gómez, and A. R. Figueiras-Vidal, "Pattern classification with missing data: a review," *Neural Computing and Applications*, 2010.

[21] O. Troyanskaya, M. Cantor, G. Sherlock, P. Brown, T. Hastie, R. Tibshirani, D. Botstein, and R. Altman, "Missing value estimation methods for dna microarrays," *Bioinformatics*, 2001.

[22] J.-F. Cai, E. J. Candès, and Z. Shen, "A singular value thresholding algorithm for matrix completion," *SIAM Journal on optimization*, 2010.

[23] E. Candes and B. Recht, "Exact matrix completion via convex optimization," *Communications of the ACM*, 2012.

[24] D. Jarrett, B. Cebere, T. Liu, A. Curth, and M. van der Schaar, "Hyperimpute: Generalized iterative imputation with automatic model selection," in *International Conference on Machine Learning*, 2022.

[25] T. Kyono, Y. Zhang, A. Bellot, and M. van der Schaar, "MIRACLE: causally-aware imputation via learning missing data mechanisms," in *Advances in Neural Information Processing Systems*, 2021.

[26] S. van Buuren and K. Groothuis-Oudshoorn, "mice: Multivariate imputation by chained equations in r," *Journal of Statistical Software*, 2011.

[27] D. J. Stekhoven and P. Bühlmann, "Missforest—non-parametric missing value imputation for mixed-type data," *Bioinformatics*, 2012.

[28] S. Van Buuren, J. P. Brand, C. G. Groothuis-Oudshoorn, and D. B. Rubin, "Fully conditional specification in multivariate imputation," *Journal of statistical computation and simulation*, 2006.

[29] J. Yoon, J. Jordon, and M. van der Schaar, "GAIN: missing data imputation using generative adversarial nets," in *Proceedings of the 35th International Conference on Machine Learning*, 2018.

[30] P. Mattei and J. Frellsen, "MIWAE: deep generative modelling and imputation of incomplete data," in *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[31] S. Yoon and S. Sull, "GAMIN: generative adversarial multiple imputation network for highly missing data," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[32] S. C. Li, B. Jiang, and B. M. Marlin, "Misgan: Learning from incomplete data with generative adversarial networks," in *International Conference on Learning Representations*, 2019.

[33] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[34] M. Villaizán-Vallelado, M. Salvatori, C. Segura, and I. Arapakis, "Diffusion models for tabular data imputation and synthetic data generation," *ACM Transactions on Knowledge Discovery from Data*, 2025.

[35] T. Liu, J. Fan, Y. Luo, N. Tang, G. Li, and X. Du, "Adaptive data augmentation for supervised learning over missing data," *Proceedings of the VLDB Endowment*, 2021.

[36] H. Zhang, L. Fang, Q. Wu, and P. Yu, "Diffputer: Empowering diffusion models for missing data imputation," in *International Conference on Learning Representations*, 2025.

[37] Y. Liu, T. Ajanthan, H. Husain, and V. Nguyen, "Self-supervision improves diffusion models for tabular data imputation," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 2024.

[38] M. A. Ahamed, Q. Ye, and Q. Cheng, "Refidiff: Progressive refinement diffusion for efficient missing data imputation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

[39] X. Huang and S. Hao, "Missing value imputation via pre-trained language models with trainable prompt and retrieval augmentation." in *VLDB Workshops*, 2024.

[40] A. Narayan, I. Chami, L. J. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, 2022.

[41] N. Tang, J. Fan, F. Li, J. Tu, X. Du, G. Li, S. Madden, and M. Ouzzani, "RPT: relational pre-trained transformer is almost all you need towards democratizing data preparation," *Proceedings of the VLDB Endowment*, 2021.

[42] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, 2017.

[43] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of Massive Datasets, 2nd Ed.* Cambridge University Press, 2014.